

BTreeVisualization.pro

```
QT += core widgets
```

```
CONFIG += c++11
```

```
TARGET = BTreeVisualization
```

```
TEMPLATE = app
```

```
SOURCES += \  
    main_qt.cpp \  
    mainwindow.cpp \  
    btreetwidget.cpp \  
    btree.cpp
```

```
HEADERS += \  
    mainwindow.h \  
    btreetwidget.h \  
    btree.h
```

```
FORMS += \  
    mainwindow.ui
```

main_qt.cpp

```
#include "mainwindow.h"  
#include <QApplication>  
#include <QTime>
```

```
int main(int argc, char *argv[])  
{  
    QApplication app(argc, argv);  
  
    app.setApplicationName("B 树可视化系统");  
    app.setApplicationVersion("1.0");  
    app.setOrganizationName("数据结构演示程序");  
  
    MainWindow window;  
    window.show();  
  
    return app.exec();  
}
```

mainwindow.cpp

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QTime>
#include <QVector>
#include <QTextCursor>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    setupUI();
    updateDisplay();
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::setupUI()
{
    // 创建并设置自定义的 BTreeWidget
    m_treeWidget = new BTreeWidget(this);
    ui->treeScrollArea->setWidget(m_treeWidget);

    // 连接信号槽
    connect(ui->insertButton, &QPushButton::clicked, this, &MainWindow::onInsertClicked);
    connect(ui->deleteButton, &QPushButton::clicked, this, &MainWindow::onDeleteClicked);
    connect(ui->searchButton, &QPushButton::clicked, this, &MainWindow::onSearchClicked);
    connect(ui->clearButton, &QPushButton::clicked, this, &MainWindow::onClearClicked);
    connect(ui->testDataButton, &QPushButton::clicked, this,
    &MainWindow::onInsertTestDataClicked);
    connect(ui->traverseButton, &QPushButton::clicked, this, &MainWindow::onTraverseClicked);

    // 允许回车键触发插入
    connect(ui->keyInput, &QLineEdit::returnPressed, this, &MainWindow::onInsertClicked);
}

void MainWindow::onInsertClicked()
{
    bool ok;
```

```

int key = ui->keyInput->text().toInt(&ok);

if (!ok) {
    QMessageBox::warning(this, "输入错误", "请输入一个有效的整数！");
    return;
}

// 检查关键字是否已经存在
if (m_tree.search(key) != nullptr) {
    showMessage(QString("数值 %1 已存在于树中！").arg(key));
    return;
}

m_tree.insert(key);
showMessage(QString("已插入数值: %1").arg(key));
updateDisplay();
ui->keyInput->clear();
ui->keyInput->setFocus();
}

void MainWindow::onDeleteClicked()
{
    bool ok;
    int key = ui->keyInput->text().toInt(&ok);

    if (!ok) {
        QMessageBox::warning(this, "输入错误", "请输入一个有效的整数！");
        return;
    }

    if (m_tree.search(key) == nullptr) {
        showMessage(QString("数值 %1 在树中不存在！").arg(key));
        return;
    }

    m_tree.remove(key);
    showMessage(QString("已删除数值: %1").arg(key));
    updateDisplay();
    ui->keyInput->clear();
    ui->keyInput->setFocus();
}

void MainWindow::onSearchClicked()
{

```

```

bool ok;
int key = ui->keyInput->text().toInt(&ok);

if (!ok) {
    QMessageBox::warning(this, "输入错误", "请输入一个有效的整数！");
    return;
}

BTreeNode* result = m_tree.search(key);
if (result != nullptr) {
    showMessage(QString("数值 %1 在树中找到了！").arg(key));
    m_treeWidget->highlightKey(key);
} else {
    showMessage(QString("数值 %1 在树中未找到！").arg(key));
    m_treeWidget->clearHighlight();
}

ui->keyInput->clear();
ui->keyInput->setFocus();
}

void MainWindow::onClearClicked()
{
    if (QMessageBox::question(this, "确认清空",
                             "您确定要清空整棵树吗？",
                             QMessageBox::Yes | QMessageBox::No) == QMessageBox::Yes) {
        m_tree.clear();
        showMessage("树已清空成功！");
        updateDisplay();
    }
}

void MainWindow::onInsertTestDataClicked()
{
    QVector<int> testKeys = {10, 20, 5, 6, 12, 30, 7, 17};

    QString insertedKeys;
    for (int key : testKeys) {
        if (m_tree.search(key) == nullptr) { // 只在还不存在时才插入
            m_tree.insert(key);
            insertedKeys += QString::number(key) + " ";
        }
    }
}

```

```

        if (!insertedKeys.isEmpty()) {
            showMessage("已插入测试数据: " + insertedKeys.trimmed());
            updateDisplay();
        } else {
            showMessage("所有测试数据都已存在于树中! ");
        }
    }
}

void MainWindow::onTraverseClicked()
{
    QString traversal = m_tree.getTraversalString();
    if (traversal.isEmpty()) {
        showMessage("树为空 - 无法进行遍历");
    } else {
        showMessage("中序遍历结果: " + traversal);
    }
}

void MainWindow::updateDisplay()
{
    m_treeWidget->setTree(&m_tree);

    // 更新状态
    QString status = m_tree.isEmpty() ? "树为空" : "树包含数据";
    ui->statusLabel->setText(status);
}

void MainWindow::showMessage(const QString &message)
{
    ui->logOutput->append(QString("[%1] %2")
                          .arg(QTime::currentTime().toString("hh:mm:ss"))
                          .arg(message));

    // 自动滚动到底部
    QTextCursor cursor = ui->logOutput->textCursor();
    cursor.movePosition(QTextCursor::End);
    ui->logOutput->setTextCursor(cursor);
}

```

BtreeWidget.cpp

```

#include "btreewidget.h"
#include <QPaintEvent>
#include <QDebug>

```

```

BTreeWidget::BTreeWidget(QWidget *parent)
    : QWidget(parent)
    , m_tree(nullptr)
    , m_highlightedKey(-1)
    , m_hasHighlight(false)
    , m_nodeWidth(80)
    , m_nodeHeight(40)
    , m_levelHeight(80)
    , m_horizontalSpacing(20)
    , m_keySpacing(25)
    , m_font("Arial", 10, QFont::Bold)
    , m_fontMetrics(m_font)
{
    setMinimumSize(600, 400);
    setStyleSheet("background-color: white;");
}

void BTreeWidget::setTree(BTree *tree)
{
    m_tree = tree;
    calculateLayout();
    update();
}

void BTreeWidget::highlightKey(int key)
{
    m_highlightedKey = key;
    m_hasHighlight = true;
    update();
}

void BTreeWidget::clearHighlight()
{
    m_hasHighlight = false;
    update();
}

void BTreeWidget::paintEvent(QPaintEvent *event)
{
    Q_UNUSED(event)

    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing);

```

```

painter.setFont(m_font);

if (!m_tree || m_tree->isEmpty()) {
    painter.setPen(Qt::gray);
    painter.drawText(rect(), Qt::AlignCenter, "B 树为空\n插入一些数据来查看可视化效果");
    return;
}

// 先画连接线，这样就在节点下面了
drawConnections(painter);

// 再画节点
for (const NodeRect &nodeRect : m_nodeRects) {
    drawNode(painter, nodeRect);
}
}

QSize BTreeWidget::sizeHint() const
{
    if (!m_tree || m_tree->isEmpty()) {
        return QSize(600, 400);
    }

    int maxX = 0;
    int maxY = 0;

    for (const NodeRect &nodeRect : m_nodeRects) {
        maxX = qMax(maxX, nodeRect.rect.right());
        maxY = qMax(maxY, nodeRect.rect.bottom());
    }

    return QSize(qMax(600, maxX + 50), qMax(400, maxY + 50));
}

void BTreeWidget::calculateLayout()
{
    m_nodeRects.clear();

    if (!m_tree || m_tree->isEmpty()) {
        return;
    }

    // 第一步：算出整棵树的总宽度
    int treeWidth = calculateSubtreeWidth(m_tree->getRoot());

```

```

// 把整棵树放在控件中间
int availableWidth = qMax(width(), 800); // 最小宽度
int startX = qMax(50, (availableWidth - treeWidth) / 2);
int nextX = startX;

layoutNodeCentered(m_tree->getRoot(), nextX, 50, 0);

// 调整控件大小以适应树
int maxX = 0;
int maxY = 0;
for (const NodeRect &nodeRect : m_nodeRects) {
    maxX = qMax(maxX, nodeRect.rect.right());
    maxY = qMax(maxY, nodeRect.rect.bottom());
}

setMinimumSize(qMax(800, maxX + 50), qMax(400, maxY + 50));
}

void BTreeWidget::layoutNodeCentered(BTreeNode *node, int &x, int y, int level)
{
    if (!node) return;

    NodeRect nodeRect;
    nodeRect.node = node;
    nodeRect.level = level;

    // 获取节点的关键字
    for (int i = 0; i < node->getKeyCount(); i++) {
        nodeRect.keys.append(node->getKey(i));
    }

    // 根据关键字数量计算节点宽度
    int nodeWidth = qMax(m_nodeWidth, nodeRect.keys.size() * m_keySpacing + 20);

    // 如果是叶子节点，直接定位
    if (node->isLeaf()) {
        nodeRect.rect = QRect(x, y, nodeWidth, m_nodeHeight);
        m_nodeRects.append(nodeRect);
        x += nodeWidth + m_horizontalSpacing;
    } else {
        // 内部节点要先摆放所有子节点
        int childY = y + m_levelHeight;
        int childStartX = x;
    }
}

```



```

// 先布局所有子节点
for (int i = 0; i <= node->getKeyCount(); i++) {
    BTreeNode *child = node->getChild(i);
    if (child) {
        layoutNodeCentered(child, x, childY, level + 1);
    }
}

// 然后把父节点放在子节点上方中间
int childrenWidth = x - childStartX - m_horizontalSpacing;
int parentX = childStartX + (childrenWidth - nodeWidth) / 2;

nodeRect.rect = QRect(parentX, y, nodeWidth, m_nodeHeight);
m_nodeRects.append(nodeRect);
}
}

void BTreeWidget::drawNode(QPainter &painter, const NodeRect &nodeRect)
{
    // 节点背景
    QRect rect = nodeRect.rect;

    if (nodeRect.node->isLeaf()) {
        painter.setBrush(QColor(220, 255, 220)); // 叶子节点用浅绿色
    } else {
        painter.setBrush(QColor(220, 220, 255)); // 内部节点用浅蓝色
    }

    painter.setPen(QPen(Qt::black, 2));
    painter.drawRect(rect);

    // 画关键字
    painter.setPen(Qt::black);
    int keyWidth = rect.width() / nodeRect.keys.size();

    for (int i = 0; i < nodeRect.keys.size(); i++) {
        int key = nodeRect.keys[i];
        QRect keyRect(rect.x() + i * keyWidth, rect.y(), keyWidth, rect.height());

        // 如果这个关键字正在被搜索，就高亮显示
        if (m_hasHighlight && key == m_highlightedKey) {
            painter.fillRect(keyRect, QColor(255, 255, 0, 128)); // 黄色高亮
        }
    }
}

```

```

// 画关键字分隔线（最后一个不画）
if (i > 0) {
    painter.drawLine(keyRect.left(), rect.top(), keyRect.left(), rect.bottom());
}

// 画关键字文本
painter.drawText(keyRect, Qt::AlignCenter, QString::number(key));
}

// 添加节点类型标识
painter.setPen(Qt::gray);
QFont smallFont = m_font;
smallFont.setPointSize(8);
painter.setFont(smallFont);

QString nodeType = nodeRect.node->isLeaf() ? "叶" : "内";
painter.drawText(rect.adjusted(2, 2, -2, -2), Qt::AlignTop | Qt::AlignRight, nodeType);

painter.setFont(m_font);
}

int BTreeWidget::calculateSubtreeWidth(BTreeNode *node)
{
    if (!node) return 0;

    int nodeWidth = qMax(m_nodeWidth, node->getKeyCount() * m_keySpacing + 20);

    if (node->isLeaf()) {
        return nodeWidth;
    }

    // 内部节点需要计算所有子节点的总宽度
    int childrenWidth = 0;
    int childCount = 0;

    for (int i = 0; i <= node->getKeyCount(); i++) {
        BTreeNode *child = node->getChild(i);
        if (child) {
            childrenWidth += calculateSubtreeWidth(child);
            childCount++;
        }
    }
}

```

```

// 子节点之间要加间距
if (childCount > 1) {
    childrenWidth += (childCount - 1) * m_horizontalSpacing;
}

// 返回节点宽度和子节点总宽度中较大的那个
return qMax(nodeWidth, childrenWidth);
}

void BTreeWidget::drawConnections(QPainter &painter)
{
    painter.setPen(QPen(Qt::darkGray, 2));

    for (const NodeRect &nodeRect : m_nodeRects) {
        if (nodeRect.node->isLeaf()) continue;

        // 找到子节点并画连接线
        for (int i = 0; i <= nodeRect.node->getKeyCount(); i++) {
            BTreeNode *child = nodeRect.node->getChild(i);
            if (!child) continue;

            // 找到子节点的矩形
            for (const NodeRect &childRect : m_nodeRects) {
                if (childRect.node == child) {
                    // 计算连接点的位置，让居中效果更好
                    QPoint parentPoint;

                    if (nodeRect.keys.size() == 1) {
                        // 单个关键字 - 从节点的左/中/右连接
                        if (i == 0) {
                            // 左子节点 - 从左四分之一处连接
                            parentPoint = QPoint(nodeRect.rect.x() + nodeRect.rect.width() / 4,
                                                    nodeRect.rect.bottom());
                        } else {
                            // 右子节点 - 从右四分之一处连接
                            parentPoint = QPoint(nodeRect.rect.x() + 3 * nodeRect.rect.width() / 4,
                                                    nodeRect.rect.bottom());
                        }
                    } else {
                        // 多个关键字 - 从合适的位置连接
                        int sectionWidth = nodeRect.rect.width() / (nodeRect.keys.size() + 1);
                        parentPoint = QPoint(nodeRect.rect.x() + (i + 1) * sectionWidth,
                                                nodeRect.rect.bottom());
                    }
                }
            }
        }
    }
}

```



```

bool BTreeNode::isUnderflow() {
    if (is_leaf) {
        return n < 1; // 叶子节点至少要有 1 个关键字
    } else {
        return n < 1; // 内部节点至少要有 1 个关键字
    }
}

```

// 中序遍历子树

```

void BTreeNode::traverse() {
    int i;
    for (i = 0; i < n; i++) {
        // 不是叶子节点的话，先遍历左边的子树
        if (!is_leaf) {
            children[i]->traverse();
        }
        // 输出当前关键字
        cout << " " << keys[i];
    }
}

```

// 处理最右边的子节点

```

if (!is_leaf) {
    children[i]->traverse();
}
}

```

// 按层次打印树结构

```

void BTreeNode::printTree(int level) {
    // 根据层次打印缩进
    for (int i = 0; i < level; i++) {
        cout << " ";
    }
}

```

// 打印节点中的所有关键字

```

cout << "[";
for (int i = 0; i < n; i++) {
    cout << keys[i];
    if (i < n - 1) cout << ",";
}
cout << "]";

```

```

if (is_leaf) {
    cout << " (叶子)";
}

```

```

        cout << endl;

// 如果有子节点，递归打印
if (!is_leaf) {
    for (int i = 0; i <= n; i++) {
        if (children[i] != nullptr) {
            children[i]->printTree(level + 1);
        }
    }
}

// 插入关键字
void BTreeNode::insert(int k) {
    if (is_leaf) {
        // 叶子节点：找到合适位置直接插入
        int i = n - 1;
        while (i >= 0 && keys[i] > k) {
            keys[i + 1] = keys[i];
            i--;
        }
        keys[i + 1] = k;
        n++;
    } else {
        // 内部节点：找到应该插入哪个子树
        int i = 0;
        while (i < n && k > keys[i]) {
            i++;
        }
        children[i]->insert(k);

        // 插入后看看子节点有没有满
        if (children[i]->isOverflow()) {
            splitChild(i);
        }
    }
}

// 分裂子节点
void BTreeNode::splitChild(int child_index) {
    BTreeNode* full_child = children[child_index];

    // 新建一个节点
    BTreeNode* new_node = new BTreeNode(full_child->is_leaf);

```

```

// 3 阶 B 树中，满节点有 3 个关键字[0,1,2]
// 分裂策略: keys[0]留在原节点, keys[1]上移到父节点, keys[2]放到新节点

// 把中间的关键字提升到父节点
int mid_key = full_child->keys[1];

// 新节点拿到右半边
new_node->keys[0] = full_child->keys[2];
new_node->n = 1;

// 如果不是叶子节点, 子节点指针也要分配
if (!full_child->is_leaf) {
    new_node->children[0] = full_child->children[2];
    new_node->children[1] = full_child->children[3];
}

// 原节点只保留左半边
full_child->n = 1; // 只留 keys[0]

// 在父节点中给新上移的关键字腾位置
for (int j = n; j > child_index; j--) {
    keys[j] = keys[j-1];
    children[j+1] = children[j];
}

// 把中间关键字和新节点插入父节点
keys[child_index] = mid_key;
children[child_index + 1] = new_node;
n++;
}

// 查找关键字
BTreeNode* BTreeNode::search(int k) {
    // 在当前节点找第一个>=k 的关键字
    int i = 0;
    while (i < n && k > keys[i]) {
        i++;
    }

    // 2. 如果找到了等于 k 的关键字, 查找成功, 返回当前节点
    if (i < n && keys[i] == k) {
        return this;
    }
}

```

```

// 3. 如果没找到，并且当前是叶节点，说明树中不存在该关键字
if (is_leaf) {
    return nullptr;
}

// 4. 如果没找到且不是叶节点，则递归地到合适的子节点中继续查找
return children[i]->search(k);
}

// 删除关键字 k
void BTreeNode::remove(int k) {
    // 在当前节点中查找关键字 k 的位置
    int i = 0;
    while (i < n && keys[i] < k) {
        i++;
    }

    if (i < n && keys[i] == k) {
        // 找到了关键字 k
        if (is_leaf) {
            // 情况 1: 关键字在叶节点中，直接删除
            for (int j = i; j < n - 1; j++) {
                keys[j] = keys[j + 1];
            }
            n--;
        } else {
            // 情况 2: 关键字在内部节点中，用左子树最大关键字替换
            int max_key = children[i]->getMaxKey();
            keys[i] = max_key;
            children[i]->remove(max_key);

            // 检查子节点是否下溢
            if (children[i]->isUnderflow()) {
                fixUnderflow(i);
            }
        }
    } else {
        // 关键字不在当前节点中
        if (is_leaf) {
            // 如果是叶节点且没找到，说明关键字不存在
            return;
        }
    }
}

```



```

        // 递归到子节点中删除
        children[i]->remove(k);

        // 检查子节点是否下溢
        if (children[i]->isUnderflow()) {
            fixUnderflow(i);
        }
    }
}

// 获取以该节点为根的子树中的最大关键字
int BTreeNode::getMaxKey() {
    if (is_leaf) {
        return keys[n - 1];
    } else {
        return children[n]->getMaxKey();
    }
}

// 从兄弟节点借关键字
void BTreeNode::borrowFromSibling(int index, bool isLeftSibling) {
    BTreeNode* child = children[index];

    if (isLeftSibling) {
        // 从左兄弟借
        BTreeNode* leftSibling = children[index - 1];

        // 将父节点的关键字下移到 child
        for (int i = child->n; i > 0; i--) {
            child->keys[i] = child->keys[i - 1];
        }
        child->keys[0] = keys[index - 1];

        // 如果不是叶节点，还要移动子节点指针
        if (!child->is_leaf) {
            for (int i = child->n + 1; i > 0; i--) {
                child->children[i] = child->children[i - 1];
            }
            child->children[0] = leftSibling->children[leftSibling->n];
        }

        // 将左兄弟的最大关键字上移到父节点
        keys[index - 1] = leftSibling->keys[leftSibling->n - 1];
    }
}

```

```

        child->n++;
        leftSibling->n--;
    } else {
        // 从右兄弟借
        BTreeNode* rightSibling = children[index + 1];

        // 将父节点的关键字下移到 child
        child->keys[child->n] = keys[index];

        // 如果不是叶节点，还要移动子节点指针
        if (!child->is_leaf) {
            child->children[child->n + 1] = rightSibling->children[0];
        }

        // 将右兄弟的最小关键字上移到父节点
        keys[index] = rightSibling->keys[0];

        // 在右兄弟中删除已移动的关键字和指针
        for (int i = 0; i < rightSibling->n - 1; i++) {
            rightSibling->keys[i] = rightSibling->keys[i + 1];
        }
        if (!rightSibling->is_leaf) {
            for (int i = 0; i < rightSibling->n; i++) {
                rightSibling->children[i] = rightSibling->children[i + 1];
            }
        }

        child->n++;
        rightSibling->n--;
    }
}

```

// 与兄弟节点合并

```

void BTreeNode::mergeWithSibling(int index) {
    BTreeNode* child = children[index];
    BTreeNode* sibling;
    bool mergeWithLeft = (index > 0);

    if (mergeWithLeft) {
        // 与左兄弟合并
        sibling = children[index - 1];

        // 将父节点的关键字下移到左兄弟
        sibling->keys[sibling->n] = keys[index - 1];
    }
}

```

```

sibling->n++;

// 将 child 的所有关键字移到左兄弟
for (int i = 0; i < child->n; i++) {
    sibling->keys[sibling->n] = child->keys[i];
    sibling->n++;
}

// 如果不是叶节点，还要移动子节点指针
if (!child->is_leaf) {
    int start_pos = sibling->n - child->n;
    for (int i = 0; i <= child->n; i++) {
        sibling->children[start_pos + i] = child->children[i];
    }
}

// 在父节点中删除已下移的关键字和 child 指针
for (int i = index - 1; i < n - 1; i++) {
    keys[i] = keys[i + 1];
}
for (int i = index; i < n; i++) {
    children[i] = children[i + 1];
}

delete child;
n--;
} else {
    // 与右兄弟合并
    sibling = children[index + 1];

    // 将父节点的关键字下移到 child
    child->keys[child->n] = keys[index];
    child->n++;

    // 将右兄弟的所有关键字移到 child
    for (int i = 0; i < sibling->n; i++) {
        child->keys[child->n] = sibling->keys[i];
        child->n++;
    }

    // 如果不是叶节点，还要移动子节点指针
    if (!child->is_leaf) {
        int start_pos = child->n - sibling->n;
        for (int i = 0; i <= sibling->n; i++) {

```

```

        child->children[start_pos + i] = sibling->children[i];
    }
}

// 在父节点中删除已下移的关键字和 sibling 指针
for (int i = index; i < n - 1; i++) {
    keys[i] = keys[i + 1];
}
for (int i = index + 1; i < n; i++) {
    children[i] = children[i + 1];
}

delete sibling;
n--;
}
}

// 修复子节点的下溢
void BTreeNode::fixUnderflow(int child_index) {
    BTreeNode* child = children[child_index];

    // 检查左兄弟是否可以借关键字
    if (child_index > 0 && children[child_index - 1]->n > 1) {
        borrowFromSibling(child_index, true);
        return;
    }

    // 检查右兄弟是否可以借关键字
    if (child_index < n && children[child_index + 1]->n > 1) {
        borrowFromSibling(child_index, false);
        return;
    }

    // 无法借关键字，需要合并
    mergeWithSibling(child_index);
}

// =====
// BTree 类的函数实现
// =====

// BTree 的构造函数
BTree::BTree() {
    root = nullptr; // 初始化为空树
}

```

```

}

// 遍历整棵树
void BTree::traverse() {
    if (root != nullptr) {
        root->traverse();
    }
    cout << endl;
}

// 打印整棵树的结构
void BTree::printTree() {
    if (root != nullptr) {
        cout << "B-Tree Structure:" << endl;
        root->printTree(0);
    } else {
        cout << "Tree is empty" << endl;
    }
}

// 查找关键字 k
BTreeNode* BTree::search(int k) {
    // 如果树为空，直接返回 null；否则从根节点开始搜索
    return (root == nullptr) ? nullptr : root->search(k);
}

// 插入一个关键字到 B 树中
void BTree::insert(int k) {
    if (root == nullptr) {
        root = new BTreeNode(true);
        root->keys[0] = k;
        root->n = 1;
        return;
    }

    // 直接插入
    root->insert(k);

    // 检查根节点是否溢出
    if (root->isOverflow()) {
        // 创建新根
        BTreeNode* new_root = new BTreeNode(false);
        new_root->children[0] = root;
        new_root->splitChild(0);
    }
}

```

```

        root = new_root;
    }
}

// 删除一个关键字
void BTree::remove(int k) {
    if (root == nullptr) {
        return;
    }

    // 检查关键字是否存在
    if (search(k) == nullptr) {
        return;
    }

    // 从根节点开始删除
    root->remove(k);

    // 检查根节点是否变为空
    if (root->n == 0) {
        BTreeNode* old_root = root;
        if (root->is_leaf) {
            // 如果根节点是叶节点且为空，整棵树变为空
            root = nullptr;
        } else {
            // 如果根节点是内部节点且为空，将其唯一的子节点作为新根
            root = root->children[0];
        }
        delete old_root;
    }
}

// 清空整棵树
void BTree::clear() {
    if (root != nullptr) {
        clearNode(root);
        root = nullptr;
    }
}

// 递归清空节点（私有辅助函数）
void BTree::clearNode(BTreeNode* node) {
    if (node != nullptr) {
        if (!node->is_leaf) {

```

```

        for (int i = 0; i <= node->n; i++) {
            clearNode(node->children[i]);
        }
    }
    delete node;
}
}

```

// Qt 可视化需要的方法 - 获取遍历字符串

```

QString BTree::getTraversalString() const {
    if (root == nullptr) {
        return "";
    }

    QString result;
    getTraversalStringHelper(root, result);
    return result.trimmed();
}

```

// 遍历辅助函数

```

void BTree::getTraversalStringHelper(BTreeNode* node, QString& result) const {
    if (node == nullptr) return;

    int i;
    for (i = 0; i < node->n; i++) {
        // 如果不是叶节点，先递归遍历左侧的子树
        if (!node->is_leaf) {
            getTraversalStringHelper(node->children[i], result);
        }
        // 然后添加当前节点的关键字
        result += QString::number(node->keys[i]) + " ";
    }

    // 处理最后一个子节点（第 n 个子节点）
    if (!node->is_leaf) {
        getTraversalStringHelper(node->children[i], result);
    }
}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

```

```

#include <QMainWindow>
#include <QMessageBox>
#include "btreewidget.h"
#include "btree.h"

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void onInsertClicked();
    void onDeleteClicked();
    void onSearchClicked();
    void onClearClicked();
    void onInsertTestDataClicked();
    void onTraverseClicked();

private:
    void setupUI();
    void updateDisplay();
    void showMessage(const QString &message);

    Ui::MainWindow *ui;

    // 自定义组件
    BTreeWidget *m_treeWidget;

    // B 树对象
    BTree m_tree;
};

#endif // MAINWINDOW_H

```

btreewidget.h

```

#ifndef BTREEWIDGET_H

```



```

#define BTREEWIDGET_H

#include <QWidget>
#include <QPainter>
#include <QVector>
#include <QPoint>
#include <QRect>
#include <QFont>
#include <QFontMetrics>
#include "btree.h"

struct NodeRect {
    QRect rect;
    BTreeNode* node;
    QVector<int> keys;
    int level;
};

class BTreeWidget : public QWidget
{
    Q_OBJECT

public:
    explicit BTreeWidget(QWidget *parent = nullptr);

    void setTree(BTree *tree);
    void highlightKey(int key);
    void clearHighlight();

protected:
    void paintEvent(QPaintEvent *event) override;
    QSize sizeHint() const override;

private:
    void calculateLayout();
    void drawNode(QPainter &painter, const NodeRect &nodeRect);
    void drawConnections(QPainter &painter);
    int calculateSubtreeWidth(BTreeNode *node);
    void layoutNodeCentered(BTreeNode *node, int &x, int y, int level);

    BTree *m_tree;
    QVector<NodeRect> m_nodeRects;
    int m_highlightedKey;
    bool m_hasHighlight;

```

```

// 布局常量 - 改为私有成员变量
int m_nodeWidth;
int m_nodeHeight;
int m_levelHeight;
int m_horizontalSpacing;
int m_keySpacing;

QFont m_font;
QFontMetrics m_fontMetrics;
};

```

```

#endif // BTREEWIDGET_H

```

btree.h

```

#pragma once // 防止重复包含
#include <QString>

// B 树节点类
class BTreeNode {
private:
    int *keys;          // 存储关键字的数组
    BTreeNode **children; // 子节点指针数组
    int n;              // 当前节点中关键字的个数
    bool is_leaf;       // 标记是否为叶子节点

    const int m = 3; // B 树的阶数
    // 3 阶 B 树：
    // 每个节点最多有 2 个关键字，3 个子节点
    // 每个节点最少有 1 个关键字

public:
    BTreeNode(bool is_leaf_flag); // 构造函数

    bool isOverflow(); // 判断节点是否关键字过多
    bool isUnderflow(); // 判断节点是否关键字过少

    // 中序遍历
    void traverse();

    // 打印树形结构
    void printTree(int level = 0);

```

```

// 插入关键字
void insert(int k);

// 分裂子节点
void splitChild(int child_index);

// 查找关键字
BTreeNode* search(int k);

// 删除关键字
void remove(int k);

// 找到子树中的最大关键字
int getMaxKey();

// 从兄弟节点借一个关键字
void borrowFromSibling(int index, bool isLeftSibling);

// 和兄弟节点合并
void mergeWithSibling(int index);

// 处理节点下溢的情况
void fixUnderflow(int child_index);

// 给界面用的接口
int getKeyCount() const { return n; }
int getKey(int index) const { return (index < n) ? keys[index] : -1; }
bool isLeaf() const { return is_leaf; }
BTreeNode* getChild(int index) const { return (index <= n && !is_leaf) ? children[index] :
nullptr; }

// 让 BTree 类可以访问私有成员
friend class BTree;
};

// B 树类
class BTree {
private:
    BTreeNode *root; // 根节点指针
    const int m = 3; // 阶数

    // 内部使用的辅助函数
    void clearNode(BTreeNode* node);
    void getTraversalStringHelper(BTreeNode* node, QString& result) const;

```

```
public:
    BTree(); // 构造函数

    // 遍历整棵树
    void traverse();

    // 打印树的结构
    void printTree();

    // 查找关键字
    BTreeNode* search(int k);

    // 插入关键字
    void insert(int k);

    // 删除关键字
    void remove(int k);

    // 清空树
    void clear();

    // 界面相关的方法
    bool isEmpty() const { return root == nullptr; }
    BTreeNode* getRoot() const { return root; }
    QString getTraversalString() const;
};
```